

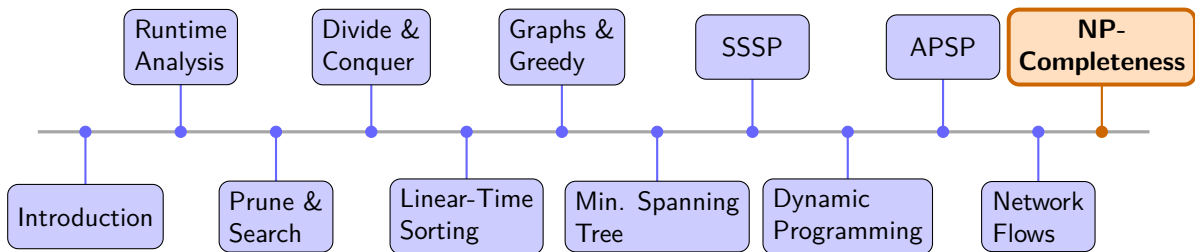
Algorithms

Dr. Mudassir Shabbir

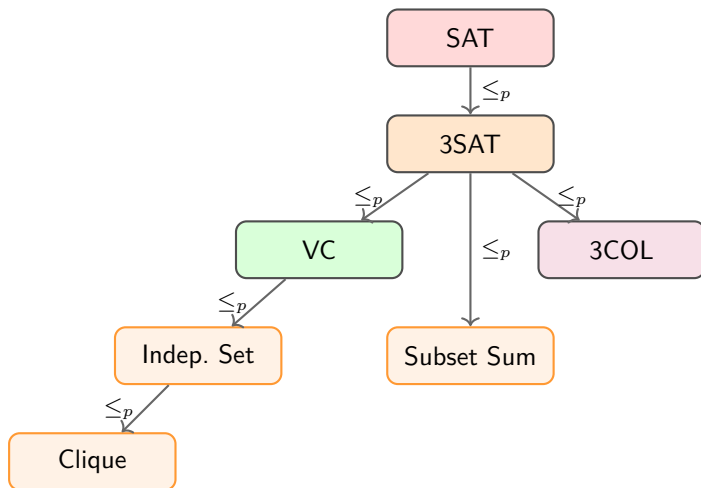
LUMS University

April 29, 2026

Recap & What's Next



Where We Are: The NPC Chain So Far



Subset Sum

Definition

Given a multiset of positive integers $S = \{s_1, s_2, \dots, s_n\}$ and a target t .

Subset Sum: Does there exist a subset $S' \subseteq S$ such that $\sum_{s \in S'} s = t$?

Example: $S = \{3, 1, 4, 2, 6\}$, $t = 9$.

Try $S' = \{3, 4, 2\}$: $3 + 4 + 2 = 9$. ✓

Example: $S = \{3, 7, 11, 2\}$, $t = 8$.

Possible sums: 3, 7, 11, 2, 10, 14, 18, 5?, ...

No subset sums to 8. ✗

Certificate for Subset Sum $\in$ NP: The subset S' . Verify $\sum_{s \in S'} s = t$ in $O(n)$. ✓

Subset Sum Example: Trying All Subsets

$S = \{1, 3, 5, 7\}$, $t = 8$.

Subset	Sum	= 8?
$\{1\}$	1	✗
$\{3\}$	3	✗
$\{5\}$	5	✗
$\{7\}$	7	✗
$\{1, 3\}$	4	✗
$\{1, 5\}$	6	✗
$\{1, 7\}$	8	✓

$S' = \{1, 7\}$ works. ✓

But the brute-force approach checks 2^n subsets—exponential in n .

This is why Subset Sum is not obviously in P. The question is whether a smarter algorithm exists.

Reduction: $3\text{SAT} \leq_p \text{Subset Sum}$

Input: 3-CNF formula φ with n variables $x_1, \dots, x_n$ and m clauses $C_1, \dots, C_m$.

High-level idea

Encode the formula as a Subset Sum instance using carefully chosen integers. Each integer represents a choice (variable true/false, clause satisfied). The target t is chosen so that a valid subset exists $\iff \varphi$ is satisfiable.

Construction. Use integers with $n + m$ decimal digits.

- For each variable x_i : two integers v_i and $\bar{v}_i$.
- For each clause C_j : two *slack* integers d_j and e_j .

Target t has:

- Digit i (variable position): $t_i = 1$ for $i = 1, \dots, n$.
- Digit $n + j$ (clause position): $t_{n+j} = 3$ for $j = 1, \dots, m$.

$3\text{SAT} \leq_p \text{Subset Sum: Integer Construction}$

Integers have $n + m$ digit positions.

Integer	Digit positions		Role
	i -th var	$n+j$ -th clause	
v_i	1 in position i	1 in $n+j$ if $x_i \in C_j$	Choose $x_i = T$
$\bar{v}_i$	1 in position i	1 in $n+j$ if $\neg x_i \in C_j$	Choose $x_i = F$
d_j	0 everywhere	1 in position $n+j$	Clause j slack
e_j	0 everywhere	1 in position $n+j$	Clause j slack

All other digit positions are 0. No carries occur (digits stay ≤ 9).

Target:

$$t = \underbrace{1\ 1\ \dots\ 1}_{n \text{ digits}} \underbrace{3\ 3\ \dots\ 3}_{m \text{ digits}}$$

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
---------	-------	-------	-------	-------	-------

Integer	x_1	x_2	x_3	C_1	C_2
---------	-------	-------	-------	-------	-------

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0

[illegible]

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1

Integer	x_1	x_2	x_3	C_1	C_2

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1
v_2	0	1	0	0	1

Integer	x_1	x_2	x_3	C_1	C_2

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1
v_2	0	1	0	0	1
$\bar{v}_2$	0	1	0	1	0

Integer	x_1	x_2	x_3	C_1	C_2

$3SAT \leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1
v_2	0	1	0	0	1
$\bar{v}_2$	0	1	0	1	0
v_3	0	0	1	1	1

Integer	x_1	x_2	x_3	C_1	C_2

$3\text{SAT} \leq_p \text{Subset Sum}$: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1
v_2	0	1	0	0	1
$\bar{v}_2$	0	1	0	1	0
v_3	0	0	1	1	1
$\bar{v}_3$	0	0	1	0	0

Integer	x_1	x_2	x_3	C_1	C_2

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1
v_2	0	1	0	0	1
$\bar{v}_2$	0	1	0	1	0
v_3	0	0	1	1	1
$\bar{v}_3$	0	0	1	0	0

Integer	x_1	x_2	x_3	C_1	C_2
Target t	1	1	1	3	3

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1
v_2	0	1	0	0	1
$\bar{v}_2$	0	1	0	1	0
v_3	0	0	1	1	1
$\bar{v}_3$	0	0	1	0	0

Integer	x_1	x_2	x_3	C_1	C_2
d_1	0	0	0	1	0
Target t	1	1	1	3	3

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1
v_2	0	1	0	0	1
$\bar{v}_2$	0	1	0	1	0
v_3	0	0	1	1	1
$\bar{v}_3$	0	0	1	0	0

Integer	x_1	x_2	x_3	C_1	C_2
d_1	0	0	0	1	0
e_1	0	0	0	1	0
Target t	1	1	1	3	3

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1
v_2	0	1	0	0	1
$\bar{v}_2$	0	1	0	1	0
v_3	0	0	1	1	1
$\bar{v}_3$	0	0	1	0	0

Integer	x_1	x_2	x_3	C_1	C_2
d_1	0	0	0	1	0
e_1	0	0	0	1	0
d_2	0	0	0	0	1
Target t	1	1	1	3	3

3SAT $\leq_p$ Subset Sum: Small Example

Formula: $\varphi = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3)$

$n = 3$ variables, $m = 2$ clauses. Digit columns: x_1, x_2, x_3, C_1, C_2 .

Integer	x_1	x_2	x_3	C_1	C_2
v_1	1	0	0	1	0
$\bar{v}_1$	1	0	0	0	1
v_2	0	1	0	0	1
$\bar{v}_2$	0	1	0	1	0
v_3	0	0	1	1	1
$\bar{v}_3$	0	0	1	0	0

Integer	x_1	x_2	x_3	C_1	C_2
d_1	0	0	0	1	0
e_1	0	0	0	1	0
d_2	0	0	0	0	1
e_2	0	0	0	0	1
Target t	1	1	1	3	3

$3\text{SAT} \leq_p \text{Subset Sum}$: Correctness

Claim

φ is satisfiable $\iff$ the Subset Sum instance has a solution.

($\Rightarrow$) SAT assignment $\Rightarrow$ Subset Sum solution.

- 1 For each variable x_i : pick v_i (if $x_i = T$) or $\bar{v}_i$ (if $x_i = F$). *Variable digits*: each i -th column sums to 1. ✓
- 2 For each clause C_j : the chosen literals contribute 1, 2, or 3 to column $n+j$. Add slack (d_j and/or e_j) to bring the column sum to 3. ✓

($\Leftarrow$) Subset Sum solution $\Rightarrow$ SAT assignment.

- Variable columns each sum to 1 $\Rightarrow$ exactly one of $v_i, \bar{v}_i$ chosen. This defines a truth assignment.
- Clause column $n+j$ sums to 3. Slack integers d_j, e_j contribute at most 2. The literals chosen contribute > 0 . Since slack alone contributes ≤ 2 , some literal in C_j must have been chosen $\Rightarrow$ clause C_j is satisfied. ✓

Subset Sum is NP-Complete

Theorem

Subset Sum is NP-complete.

Step 1 — Subset Sum $\in$ NP.

Certificate: the chosen subset S' . Verify $\sum_{s \in S'} s = t$ in $O(n)$. ✓

Step 2 — Subset Sum is NP-hard.

We showed $3SAT \leq_p$ Subset Sum. Since 3SAT is NP-hard:

$$\forall L \in \text{NP} : L \leq_p 3SAT \leq_p \text{Subset Sum}$$

By transitivity, Subset Sum is NP-hard. ✓

Approximation Algorithms

When Exact is Impossible, How Close Can We Get?

The Big Picture: Living with NP-Hardness

We have proved dozens of problems NP-complete. So what do we actually *do*?

Three main coping strategies

- ➊ **Exact algorithms for small input:** branch-and-bound, ILP solvers, FPT.
- ➋ **Heuristics:** no guarantee, but fast and often good in practice.
- ➌ **Approximation algorithms:** polynomial time, *provable* quality guarantee.

Today: strategy 3. We want algorithms that run in $O(n^k)$ and return a solution whose objective value is *within a factor* of optimal.

Optimization Problems, Not Decision Problems

NP-completeness was defined for **decision problems** (yes/no answers).

Approximation algorithms operate on their **optimization counterparts**.

Decision problem	→	Optimization problem
Does a VC of size $\leq k$ exist?	→	Find the <i>minimum</i> vertex cover.
Does a tour of cost $\leq B$ exist?	→	Find the <i>shortest</i> Hamiltonian tour.
Can m items of total value $\geq V$ fit?	→	<i>Maximize</i> value in the knapsack.
Can all elements be covered by $\leq k$ sets?	→	Find the <i>smallest</i> set cover.

We compare against OPT, the *true* optimum, even though we cannot compute it efficiently.

Optimization Problem: Formal View

An optimization problem is specified by:

- An instance I (input data).
- A set of valid (feasible) solutions $\mathcal{F}(I)$.
- An objective function $f_I(S)$ that assigns a numeric value to each feasible solution $S \in \mathcal{F}(I)$.

Objective Value of a Valid Solution

For any valid solution S , its **objective value** is exactly the number $f_I(S)$.

- Minimization: smaller $f_I(S)$ is better.
- Maximization: larger $f_I(S)$ is better.

Examples. Vertex Cover: $f_I(S) = |S|$ (minimize). Knapsack: $f_I(S) = \sum_{i \in S} v_i$ (maximize).

The Central Challenge: Bounding OPT

We want to prove $A(I) \leq \alpha \cdot \text{OPT}(I)$. The catch:

We cannot compute OPT: it is NP-hard!

Strategy: find a *lower bound* on OPT (for minimization)

This is the core skill: find the *right* structural lower bound for the problem at hand.

Approximation Ratio?

Minimization problem

Algorithm A is a α -**approximation** ($\alpha \geq 1$) if for every instance I :

$$A(I) \leq \alpha \cdot \text{OPT}(I).$$

Maximization problem

Algorithm A is a α -**approximation** ($\alpha \geq 1$) if for every instance I :

$$A(I) \geq \frac{\text{OPT}(I)}{\alpha} \quad (\text{equivalently, } \text{OPT}(I) \leq \alpha A(I)).$$

Approximation Ratio Classes

- Constant ratio: e.g. 2, $3/2$, $8/7$.
- Ratio $1 + \varepsilon$ for any $\varepsilon > 0$: **PTAS** (Polynomial-Time Approximation Scheme).
- Ratio $1 + \varepsilon$ with runtime polynomial in both n and $1/\varepsilon$: **FPTAS**.
- $O(\log n)$ ratio: e.g. Set Cover.
- No *useful* approximation unless $P = NP$: e.g. Clique.

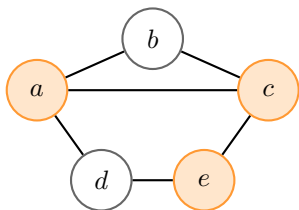
Vertex Cover: Reminder

Problem

Given an undirected graph $G = (V, E)$, find a **minimum vertex cover**: a smallest set $C \subseteq V$ such that every edge has at least one endpoint in C .

NP-hard. No poly-time exact algorithm (unless $P = NP$).

Example.



Lower bound (UGC): Cannot do better than 2.

Matching: Definitions

Matching

A **matching** in a graph $G = (V, E)$ is a set $M \subseteq E$ of edges such that no two edges in M share an endpoint.

Maximal Matching

A matching M is **maximal** if no edge can be added to M without violating the matching property (i.e., M is not a subset of any larger matching).

Maximum Matching

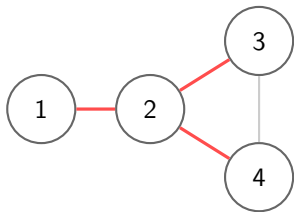
A matching M is **maximum** if it has the largest possible number of edges among all matchings. (Maximum $\Rightarrow$ maximal, but not vice versa.)

Perfect Matching

A matching M is **perfect** if every vertex in V is matched, i.e., $|M| = |V|/2$.

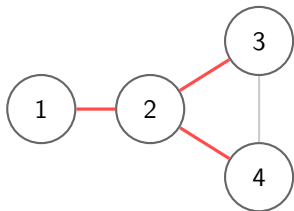
Is This a Matching?

Candidate A. Highlighted:
 $\{(1, 2), (2, 3), (2, 4)\}$.



Is This a Matching?

Candidate A. Highlighted:
 $\{(1, 2), (2, 3), (2, 4)\}$.

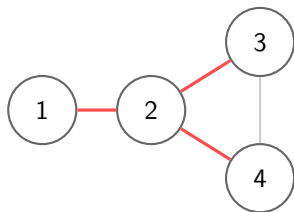


X Not a matching.

Vertex 2 is shared by all three edges.

Is This a Matching?

Candidate A. Highlighted:
 $\{(1, 2), (2, 3), (2, 4)\}$.



X Not a matching.

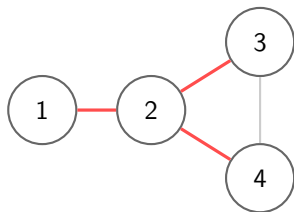
Vertex 2 is shared by all three edges.

Candidate B. Highlighted: $\{(1, 2), (3, 4)\}$.



Is This a Matching?

Candidate A. Highlighted:
 $\{(1, 2), (2, 3), (2, 4)\}$.



✗ Not a matching.

Vertex 2 is shared by all three edges.

Candidate B. Highlighted: $\{(1, 2), (3, 4)\}$.



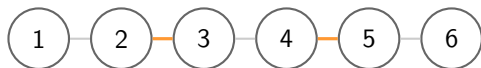
✓ A matching.

No two highlighted edges share a vertex.

Maximal vs. Maximum

Graph: path $1-2-3-4-5-6$.

Candidate A: $M_A = \{(2, 3), (4, 5)\}$.

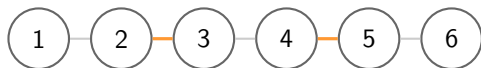


Matching? Maximum? Maximal? Perfect?

Maximal vs. Maximum

Graph: path $1-2-3-4-5-6$.

Candidate A: $M_A = \{(2, 3), (4, 5)\}$.



Matching? Maximum? Maximal? Perfect?

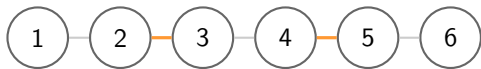
✓ Matching ✓ Maximal

✗ Not maximum ($|M_A| = 2$) ✗ Not perfect

Maximal vs. Maximum

Graph: path 1–2–3–4–5–6.

Candidate A: $M_A = \{(2, 3), (4, 5)\}$.

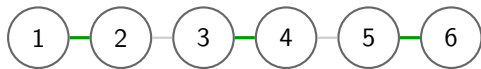


Matching? Maximum? Maximal? Perfect?

✓ Matching ✓ Maximal

✗ Not maximum ($|M_A| = 2$) ✗ Not perfect

Candidate B: $M_B = \{(1, 2), (3, 4), (5, 6)\}$.

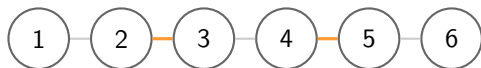


Matching? Maximum? Maximal? Perfect?

Maximal vs. Maximum

Graph: path 1–2–3–4–5–6.

Candidate A: $M_A = \{(2, 3), (4, 5)\}$.

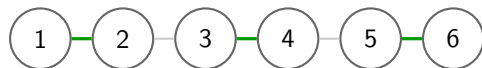


Matching? Maximum? Maximal? Perfect?

✓ Matching ✓ Maximal

✗ Not maximum ($|M_A| = 2$) ✗ Not perfect

Candidate B: $M_B = \{(1, 2), (3, 4), (5, 6)\}$.



Matching? Maximum? Maximal? Perfect?

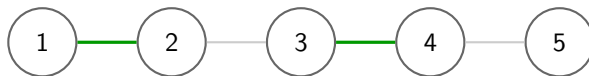
✓ Matching ✓ Maximal

✓ Maximum ($|M_B| = 3$) ✓ Perfect

Note: maximal $\nRightarrow$ maximum. But maximum $\Rightarrow$ maximal.

Maximum but Not Perfect

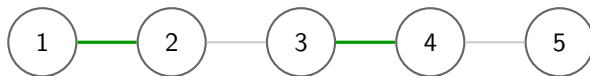
Graph: path $1-2-3-4-5$. Highlighted matching: $M = \{(1, 2), (3, 4)\}$.



Is M a matching? Is it maximum? Is it maximal? Is it perfect?

Maximum but Not Perfect

Graph: path $1-2-3-4-5$. Highlighted matching: $M = \{(1, 2), (3, 4)\}$.



Is M a matching? Is it maximum? Is it maximal? Is it perfect?

✓ **Matching** (no shared endpoints).

✓ **Maximum:** size 3 would need 6 endpoints, impossible with only 5 vertices.

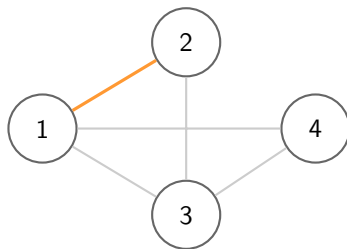
✓ **Maximal:** only free vertex is 5, and $(4, 5)$ would conflict with 4.

✗ **Not perfect:** vertex 5 is unmatched.

Why? A perfect matching requires $|V|$ even. Here $|V| = 5$, so some vertex is always left out.

Is This Maximal?

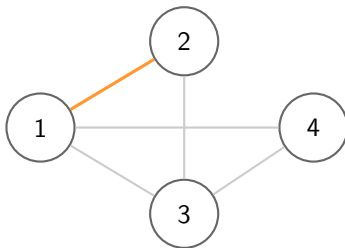
Graph: $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (1, 3), (1, 4), (2, 3), (3, 4)\}$. Highlighted:
 $M = \{(1, 2)\}$.



Is $M = \{(1, 2)\}$ a maximal matching?

Is This Maximal?

Graph: $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (1, 3), (1, 4), (2, 3), (3, 4)\}$. Highlighted:
 $M = \{(1, 2)\}$.



Is $M = \{(1, 2)\}$ a maximal matching?

X Not maximal. Edge (3, 4) can still be added: vertices 3 and 4 are both free.

Matching Number and Vertex Cover: The Key Inequality

Fundamental Inequality

For any graph G , any matching M , and any vertex cover C :

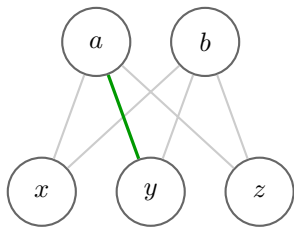
$$|M| \leq |C|.$$

Why? Each edge in M must be covered by C , and no single vertex can cover two matching edges (they share no endpoint). So C needs at least one distinct vertex per matching edge.

This gives us a nice bound on OPT_{VC} .

Matching Number and Vertex Cover: Example

Graph: $K_{2,3}$ (complete bipartite: top $\{a, b\}$, bottom $\{x, y, z\}$).



Max matching: $M = \{(a, y), (b, x)\}$, $|M| = 2$.
(Add (b, x) to the picture.)

Our 2-approx VC: take both endpoints of each matching edge $\Rightarrow C = \{a, b, x, y\}$, $|C| = 4$.

Optimal VC: $C^* = \{a, b\}$: both top vertices cover all 6 edges! $|C^*| = 2$.

Ratio: $4/2 = 2$. This is the *worst case* for our algorithm on this graph.

König's theorem (bipartite only):
 $\nu(G) = \tau(G)$, i.e., max matching = min vertex cover. Here $\nu = \tau = 2$.

Vertex Cover: 2-Approximation via Maximal Matching

Algorithm APPROXVC

```
1:  $C \leftarrow \emptyset, E' \leftarrow E$ 
2: while  $E' \neq \emptyset$  do
3:   Pick any edge  $(u, v) \in E'$ 
4:    $C \leftarrow C \cup \{u, v\}$ 
5:   Remove all edges incident to  $u$  or  $v$  from  $E'$ 
6: end while
7: return  $C$ 
```

Key observation. The selected edges form a *maximal matching* M . No two selected edges share an endpoint, so $|M|$ edges give $|C| = 2|M|$ vertices.

Theorem: APPROXVC is a 2-approximation.

$\text{OPT} \geq |M|$ (matching lower bound), so $|C| = 2|M| \leq 2 \cdot \text{OPT}$.

Vertex Cover 2-Approx: Worked Example

Graph: K_4 (complete on $\{1, 2, 3, 4\}$).

Step 1. Pick edge $(1, 2)$. Add 1, 2 to C .

Remove all edges incident to 1 or 2.

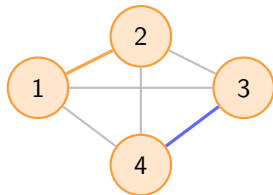
Step 2. Remaining edges: $\{(3, 4)\}$. Pick $(3, 4)$.

Add 3, 4 to C . Done.

$C = \{1, 2, 3, 4\}$, $|C| = 4$.

OPT = 3 (e.g. $\{1, 2, 3\}$ covers all 6 edges).

Ratio achieved: $4/3 < 2$. ✓



Orange/blue = matched edges.

All 4 vertices added to C .

Worst case for this algorithm: a perfect matching on $2k$ vertices: gives $C = V$ but OPT = k .

Load Balancing: Problem Definition

Problem: Makespan Minimization ($P||C_{\max}$)

Given n jobs with processing times $p_1, p_2, \dots, p_n > 0$ and m identical parallel machines.

Goal: Assign each job to a machine to **minimize the makespan**

$$C_{\max} = \max_i \sum_{j \text{ on machine } i} p_j.$$

Two lower bounds on OPT:

$$\text{OPT} \geq \frac{1}{m} \sum_{j=1}^n p_j \quad (\text{average load})$$

$$\text{OPT} \geq \max_j p_j \quad (\text{longest job must run somewhere})$$

These two simple bounds are the foundation for most approximation analyses.

Load Balancing: List Scheduling (Greedy)

Algorithm LISTSCHEDULE

Process jobs $1, 2, \dots, n$ in any order.

Assign each job to the machine with the **current smallest load**.

Example. $m = 3$ machines, jobs: 3, 4, 2, 5, 1, 2.

Job	Time	Assigned to	Machine Loads
3	3	M1	[3, 0, 0]
4	4	M2	[3, 4, 0]
2	2	M3	[3, 4, 2]
5	5	M3	[3, 4, 7]
1	1	M1	[4, 4, 7]
2	2	M1 or M2	[6, 6, 7]

$C_{\max} = 7$. $\text{OPT} = 6$ (assign $\{5, 1\}, \{4, 2\}, \{3, 2\}$). Ratio: $7/6 < 2$. ✓

List Scheduling: 2-Approximation Analysis

Theorem

LISTSCHEDULE is a 2-approximation for makespan minimization.

Proof.

Let j^* be the job that *finishes last* (determines $C_{\max}$).

Let T be the load on j^* 's machine *just before* j^* was assigned.

By greedy choice, every other machine had load $\geq T$ when j^* was assigned.

Summing over all m machines:

$$m \cdot T \leq \sum_{j \neq j^*} p_j \leq \sum_{j=1}^n p_j \leq m \cdot \text{OPT}.$$

So $T \leq \text{OPT}$. Therefore:

$$C_{\max} = T + p_{j^*} \leq \text{OPT} + \text{OPT} = 2 \text{OPT}.$$

Load Balancing: LPT Rule

Algorithm LPT (Longest Processing Time first)

Sort jobs so that $p_1 \geq p_2 \geq \dots \geq p_n$.

Then run `LISTSCHEDULE` in this sorted order.

Theorem (Graham 1969)

LPT is a $\frac{3}{2}$ -approximation for makespan minimization.

Intuition. Placing large jobs first leaves smaller jobs to fill gaps: less room for bad imbalance.

Key lemma (simpler). Under LPT, the last-finishing job j^* satisfies $p_{j^*} \leq \text{OPT}/2$ (when $n > m$). Plugging into the List Scheduling analysis gives $C_{\max} \leq \text{OPT} + \text{OPT}/2 = 3\text{OPT}/2$.

LPT: Why the $3/2$ Bound Holds

Claim: Under LPT with $n > m$, the last job j^* satisfies $p_{j^*} \leq \text{OPT}/2$.

Proof sketch.

If $n \leq m$, every job can go to its own machine, so LPT is optimal. So assume $n > m$.

Under LPT, the last-finishing job is among the smaller jobs. At least $m + 1$ jobs have size $\geq p_{j^*}$. By pigeonhole, in any schedule some machine gets at least two of these jobs, so

$$\text{OPT} \geq 2p_{j^*}.$$

Hence $p_{j^*} \leq \text{OPT}/2$. Combining with the List Scheduling decomposition $C_{\max} = T + p_{j^*}$ and $T \leq \text{OPT}$ gives

$$C_{\max} \leq \text{OPT} + \frac{\text{OPT}}{2} = \frac{3}{2} \text{OPT}.$$

✓

LPT: Worked Example Showing Improvement

$m = 3$ machines. Jobs (sorted): 7, 5, 5, 3, 3, 2.

List Scheduling (arbitrary order 2, 3, 3, 5, 5, 7):

Job	Time	Machine
2	2	M1
3	3	M2
3	3	M3
5	5	M1
5	5	M2
7	7	M3

Loads: [7, 8, 10]. $C_{\max} = 10$.

OPT = 9 (cannot do better: total load = 25, $\lceil 25/3 \rceil = 9$, job of size 7).

LPT (sorted 7, 5, 5, 3, 3, 2):

Job	Time	Machine
7	7	M1
5	5	M2
5	5	M3
3	3	M2
3	3	M3
2	2	M1

Loads: [9, 8, 8]. $C_{\max} = 9$.

Set Cover: Problem

Problem

Universe $U = \{1, \dots, m\}$, collection $\mathcal{S} = \{S_1, \dots, S_n\}$ of subsets of U .

Goal: Find the smallest subcollection $\mathcal{C} \subseteq \mathcal{S}$ such that $\bigcup_{S \in \mathcal{C}} S = U$.

NP-hard (generalizes Vertex Cover).

Example. $U = \{1, 2, 3, 4, 5, 6\}$, four available sets:

$$S_1 = \{1, 2, 3, 4\}, \quad S_2 = \{2, 3, 5\}, \quad S_3 = \{4, 5, 6\}, \quad S_4 = \{1, 6\}.$$

Can we cover U with 2 sets? Yes: $\mathcal{C} = \{S_1, S_3\}$ covers $\{1, 2, 3, 4\} \cup \{4, 5, 6\} = U$. ✓

Lower bound on OPT. Any single set covers $\leq m$ elements, so $\text{OPT} \geq \lceil m / \max_i |S_i| \rceil$.

Set Cover: Greedy Algorithm

Algorithm GREEDYCOVER

```
1:  $\mathcal{C} \leftarrow \emptyset$ ,  $R \leftarrow U$    (remaining uncovered elements)
2: while  $R \neq \emptyset$  do
3:   Pick  $S_i \in \mathcal{S}$  maximizing  $|S_i \cap R|$ 
4:    $\mathcal{C} \leftarrow \mathcal{C} \cup \{S_i\}$ ,  $R \leftarrow R \setminus S_i$ 
5: end while
6: return  $\mathcal{C}$ 
```

Greedy trace. $U = \{1, \dots, 6\}$, sets $S_1 = \{1, 2, 3, 4\}$, $S_2 = \{2, 3, 5\}$, $S_3 = \{4, 5, 6\}$, $S_4 = \{1, 6\}$:

Round	Set picked	Elements covered	$ R $ remaining
1	$S_1 = \{1, 2, 3, 4\}$	4 new	2
2	$S_3 = \{4, 5, 6\}$	2 new	0

Set Cover: $\ln |U|$ -Approximation Analysis

Theorem

GREEDYCOVER is an H_m -approximation, where $H_m = 1 + \frac{1}{2} + \cdots + \frac{1}{m} \approx \ln m$.

Proof sketch.

Let $\text{OPT} = k^*$. At any point, the k^* optimal sets together cover all remaining elements, so the *best* single set covers at least a $1/k^*$ fraction of R .

If $|R| = r$ before a round, greedy picks $\geq r/k^*$ elements. After t rounds:

$$|R| \leq m \left(1 - \frac{1}{k^*}\right)^t \leq m e^{-t/k^*}.$$

Setting $|R| < 1$ (i.e. $R = \emptyset$) requires $t \leq k^* \ln m = \text{OPT} \cdot \ln m$. ✓

Set Cover: Tightness and Hardness

Tight example. $m = 2^k$ elements. Arrange sets so each set covers exactly half the remaining uncovered elements. Greedy needs $k = \log_2 m$ rounds; if $\text{OPT} = 1$ (one big set covers all), $\text{ratio} = \log m = H_m$.

Hardness (Feige 1998)

No polynomial-time algorithm achieves ratio $(1 - \varepsilon) \ln m$ for any $\varepsilon > 0$, unless

$$\text{NP} \subseteq \text{DTIME}(n^{O(\log \log n)}).$$

Conclusion. Greedy is essentially optimal for Set Cover: we cannot do meaningfully better in polynomial time.

Set Cover: Worked Example with Bad Greedy Choice

Instance where greedy is suboptimal (but still $\leq \ln m \cdot \text{OPT}$).

$U = \{1, 2, 3, 4, 5, 6\}$, $\text{OPT} = 2$.

$$S_1 = \{1, 2, 3, 4\}, S_2 = \{1, 2, 5, 6\}, S_3 = \{3, 4, 5, 6\}.$$

OPT: $\{S_1, S_3\}$ or $\{S_1, S_2\}$ or $\{S_2, S_3\}$: each covers U in 2 sets.

Greedy (breaking ties arbitrarily):

- 1 Pick S_1 (covers 4). $R = \{5, 6\}$.
- 2 Pick S_2 or S_3 (covers 2). $R = \emptyset$.

Greedy also gets 2 sets: optimal here.

Harder bad example. $U = \{1, \dots, 6\}$. $S_1 = \{1, 2, 3\}$, $S_2 = \{4, 5, 6\}$, $S_3 = \{1, 4\}$, $S_4 = \{2, 5\}$, $S_5 = \{3, 6\}$.

OPT = 2: $\{S_1, S_2\}$. Greedy might pick S_3, S_4, S_5 (each size 2) $\Rightarrow$ 3 sets.

Ratio = $3/2 \leq H_6 \approx 2.45$. ✓

Metric TSP: Problem

Metric TSP

Given n cities with pairwise distances $d_{ij} \geq 0$ satisfying the **triangle inequality**:

$$d_{ij} \leq d_{ik} + d_{kj} \quad \forall i, j, k.$$

Goal: Find a minimum-cost Hamiltonian cycle (tour).

Why the triangle inequality?

Without it, no constant-factor approximation exists (unless $P = NP$).

Reduction: given any graph G , set $d_{uv} = 1$ if $(u, v) \in E$, else $d_{uv} = \alpha n + 1$. A α -approximation would decide Hamiltonian Cycle: which is NP-hard.

So the triangle inequality is not just a convenience: it is *necessary* for any useful approximation guarantee.

Metric TSP: Known Results

Algorithm	Ratio	Key tool
MST + double tree	2	MST, Euler tour, shortcut
Christofides–Serdyukov (1976)	$3/2$	MST + min-weight matching
Karlin–Klein–Oveis Gharan (2020)	$3/2 - 10^{-36}$	Random spanning trees

Open problem. The true approximability threshold for metric TSP is unknown. No lower bound better than $123/122$ is known (assuming $P \neq NP$).

Closing the gap between $3/2$ and the best lower bound is one of the most celebrated open problems in combinatorial optimization.

Metric TSP: 2-Approximation via MST

Algorithm MST_{TOUR}

- 1 Compute a minimum spanning tree T of the complete graph.
- 2 **Double** every edge of T to get an Eulerian multigraph T' (all degrees even).
- 3 Find an **Euler tour** of T' (visits every edge exactly once).
- 4 **Shortcut** repeated vertices: traverse the Euler tour, skipping already-visited cities.

Theorem: MST_{TOUR} is a 2-approximation.

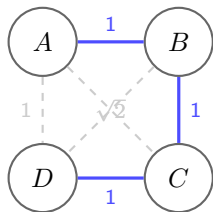
Analysis:

- Any tour $\Rightarrow$ spanning connected subgraph $\Rightarrow$ MST cost $\leq$ OPT.
- Doubling gives Euler tour of cost $2 \cdot w(T) \leq 2 \cdot \text{OPT}$.
- Shortcuts never increase cost (triangle inequality): shortcut cost $\leq$ Euler tour cost.

Therefore: MST_{TOUR} cost $\leq 2 \cdot \text{OPT}$. ✓

MST Tour: Worked Example

4 cities on a grid:



MST (blue): $A-B-C-D$, cost 3.

Double MST edges: each edge appears twice.

Euler tour: $A-B-C-D-C-B-A$ (cost 6).

Shortcut repeated B, C : Tour $A-B-C-D-A$, cost $1 + 1 + 1 + 1 = 4$.

OPT = 4 (same tour). ✓

Ratio = 1 here (we got OPT).

In general: shortcuts exploit triangle inequality; ratio is at most 2.

Christofides Algorithm: 3/2-Approximation

Observation. The MST doubling step is wasteful. Can we get a cheaper Eulerian extension?

Algorithm CHRISTOFIDES (1976)

- 1 Compute minimum spanning tree T .
- 2 Let O = set of odd-degree vertices in T . ($|O|$ is always even.)
- 3 Compute a **minimum-weight perfect matching** M on the vertices of O .
- 4 Combine $T \cup M$: all degrees now even: it is Eulerian.
- 5 Find an Euler tour; shortcut repeated vertices.

Theorem: Christofides gives a 3/2-approximation.

$w(T) \leq \text{OPT}$. The odd vertices O can be split into two matchings using alternate edges of the OPT tour restricted to O ; the cheaper matching costs $\leq \text{OPT}/2$. So total cost $\leq \text{OPT} + \text{OPT}/2 = 3\text{OPT}/2$.

Christofides: Matching on Odd Vertices

Why $w(M) \leq \text{OPT}/2$.

Let $O = \{v_1, v_2, \dots, v_{2k}\}$ be the odd-degree vertices in T , appearing in this cyclic order on the optimal tour.

The OPT tour restricted to O gives two perfect matchings:

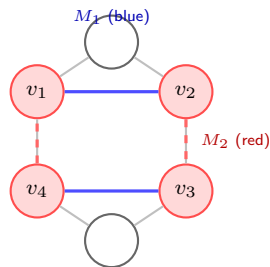
$$M_1 = \{(v_1, v_2), (v_3, v_4), \dots\}, \quad M_2 = \{(v_2, v_3), (v_4, v_5), \dots\}$$

By triangle inequality, $w(M_1) + w(M_2) \leq w(\text{OPT})$.

So $\min(w(M_1), w(M_2)) \leq \text{OPT}/2$.

The minimum perfect matching is at most this:

$$w(M) \leq \frac{\text{OPT}}{2}.$$



Red vertices = odd-degree in T .

PTAS and FPTAS: Definitions

Polynomial-Time Approximation Scheme (PTAS)

A family of algorithms $\{A_\varepsilon\}_{\varepsilon>0}$ where for each fixed $\varepsilon > 0$, algorithm A_ε is a $(1 + \varepsilon)$ -approximation running in polynomial time in n .

Runtime *may be exponential* in $1/\varepsilon$: e.g. $O(n^{1/\varepsilon})$.

Fully Polynomial-Time Approximation Scheme (FPTAS)

Same as PTAS, but the runtime must be polynomial in **both** n **and** $1/\varepsilon$.

Example: $O(n^3/\varepsilon^2)$ is FPTAS. But $O(n^{1/\varepsilon})$ is not.

Hierarchy (strict inclusions):

$\text{FPTAS} \subsetneq \text{PTAS} \subsetneq \text{constant approx} \subsetneq \text{poly approx}.$

PTAS and FPTAS: Examples

Class	Problem	Note
FPTAS	0-1 Knapsack	Scaling DP
FPTAS	Subset Sum	Special case of Knapsack
FPTAS	Scheduling $P C_{\max}$	Fixed m machines
PTAS (not FPTAS)	Euclidean TSP	Arora 1998
PTAS (not FPTAS)	Max- k -SAT	Randomized
No PTAS	Set Cover $< \ln n$	Feige 1998
No PTAS	Clique	$n^{1-\epsilon}$ hard
No constant	General TSP	Ham-Cycle reduction

Takeaway. FPTAS is the gold standard: you can get arbitrarily close to optimal in truly polynomial time. Most NP-hard problems do not admit an FPTAS.

Knapsack: FPTAS via Scaling

Problem

n items, weights w_i , values v_i ; knapsack capacity W .

Goal: Maximize $\sum_{i \in S} v_i$ subject to $\sum_{i \in S} w_i \leq W$.

Exact DP: $O(nV)$ where $V = \sum v_i$. Pseudo-polynomial: can be huge.

Algorithm KNAPSACKFPTAS(ϵ)

- 1 Let $v_{\max} = \max_i v_i$.
- 2 Scale values: $\hat{v}_i = \left\lfloor \frac{v_i \cdot n}{\epsilon \cdot v_{\max}} \right\rfloor$.
- 3 Run exact DP on scaled values $\hat{v}_i$ (now all $\leq n/\epsilon$).
- 4 Return the selected subset.

Runtime: $O(n \cdot \hat{V}) = O\left(\frac{n^3}{\epsilon}\right)$. Polynomial in n and $1/\epsilon$. ✓

Knapsack FPTAS: Correctness Analysis

Why ratio is $1/(1 + \varepsilon)$?

Let $K = \frac{\varepsilon \cdot v_{\max}}{n}$. Then $\hat{v}_i = \lfloor v_i/K \rfloor$.

For any item i : $v_i/K - 1 \leq \hat{v}_i \leq v_i/K$.

Let S^* = optimal solution (exact values), $\hat{S}$ = FPTAS solution (scaled values).

Since $\hat{S}$ is optimal for scaled values: $\sum_{i \in \hat{S}} \hat{v}_i \geq \sum_{i \in S^*} \hat{v}_i$.

Translating back (at most n items, each scaled down by < 1):

$$\sum_{i \in \hat{S}} v_i \geq K \sum_{i \in \hat{S}} \hat{v}_i \geq K \sum_{i \in S^*} \hat{v}_i \geq \sum_{i \in S^*} v_i - nK = \text{OPT} - \varepsilon \cdot v_{\max} \geq (1 - \varepsilon) \text{OPT}.$$

So FPTAS achieves at least $(1 - \varepsilon)\text{OPT}$ in $O(n^3/\varepsilon)$ time. ✓

Max-3SAT: Problem and Algorithm

Max-3SAT (Optimization)

Given a 3-CNF formula with m clauses (each clause has exactly 3 distinct literals).

Goal: Find an assignment that maximizes the number of satisfied clauses.

Algorithm: Random Assignment

Set each variable *independently* and *uniformly* at random:

$$\Pr[x_i = \text{True}] = \Pr[x_i = \text{False}] = 1/2.$$

Simplicity check. This is as simple as an algorithm can be: no problem structure used at all. Yet it achieves the best possible approximation ratio.

Max-3SAT: Analysis

How many clauses does random assignment satisfy?

A clause is *unsatisfied* only when all 3 of its literals evaluate to *false* simultaneously.

$$\Pr[\text{clause unsatisfied}] = \left(\frac{1}{2}\right)^3 = \frac{1}{8}.$$

$$\Pr[\text{clause satisfied}] = 1 - \frac{1}{8} = \frac{7}{8}.$$

By linearity of expectation, over all m clauses:

$$\mathbf{E}[\# \text{ satisfied clauses}] = \frac{7}{8} m \geq \frac{7}{8} \text{OPT}.$$

Theorem

Random assignment is a $7/8$ -approximation for Max-3SAT in expectation.

Max-3SAT: Derandomization

Can we make the algorithm deterministic?

Method of Conditional Expectations

Assign variables one by one. At each step, choose the value of x_i that keeps the *conditional expected* number of satisfied clauses $\geq 7m/8$.

Since the expectation starts at $\geq 7m/8$ and never drops below it, the final assignment satisfies $\geq 7m/8$ clauses deterministically.

This gives a *deterministic* polynomial-time $7/8$ -approximation. ✓

Max-3SAT: Tightness

Is $7/8$ the best possible?

Håstad's Inapproximability (1997)

For any $\varepsilon > 0$, it is NP-hard to approximate Max-3SAT to within a ratio of $7/8 + \varepsilon$.

Tight example. Take all $2^3 = 8$ possible 3-literal clauses over variables $\{x, y, z\}$:

$$(x \vee y \vee z), (x \vee y \vee \neg z), \dots, (\neg x \vee \neg y \vee \neg z).$$

Every truth assignment satisfies exactly 7 of the 8 clauses: so $\text{OPT}/m = 7/8$ exactly.

Conclusion. The trivial random algorithm is *essentially optimal*. No polynomial algorithm can do better.

Contrast. Max-2SAT: random gives $3/4$, tight by inapproximability; yet exact 2SAT is in P.

Inapproximability: Gap Reductions

So far: *upper bounds*: algorithms achieving ratio α .

Now: *lower bounds*: proving no poly-time algorithm can beat ratio α (unless $P = NP$).

Gap Reduction

To show no α -approximation exists for problem B , reduce from a hard problem A :

- **YES** instance of $A \Rightarrow B$ has a solution of value $\geq V$.
- **NO** instance of $A \Rightarrow$ every solution of B has value $< V/\alpha$.

A α -approximation for B would distinguish these two cases, solving A exactly: contradiction.

Example. General TSP inapproximability (proved earlier) is a gap reduction from Ham-Cycle with gap αn vs n .

Inapproximability: The PCP Theorem

Most inapproximability results for non-trivial ratios rely on a deep theorem:

PCP Theorem (Arora–Safra, Arora–Lund–Motwani–Sudan–Szegedy, 1992)

$$\text{NP} = \text{PCP}(O(\log n), O(1)).$$

Every NP problem has a *probabilistically checkable proof*: a verifier that reads only $O(\log n)$ random bits and $O(1)$ proof bits, yet accepts correct proofs with probability 1 and rejects incorrect proofs with probability $\geq 1/2$.

Why it implies inapproximability. The PCP theorem implies that Max-3SAT is hard to approximate above $7/8$, which then propagates via reductions to Vertex Cover, Set Cover, Clique, and many others.

It is one of the most influential results in theoretical computer science.

Inapproximability: Key Results

Problem	Best ratio	Inapproximability
Vertex Cover	$2 - O(1/\sqrt{\log n})$	$\geq 10\sqrt{5} - 21 \approx 1.36$ (UGC: 2)
Set Cover	$\ln n + O(1)$	$\leq (1 - \varepsilon) \ln n$ (unless $P \neq NP$)
Clique	$n^{1-\varepsilon}$	$n^{1-\varepsilon}$ ($P \neq NP$)
Max-3SAT	$7/8$	$7/8 + \varepsilon$
Metric TSP	$3/2$	No $\alpha < 123/122$ known
General TSP	:	No constant factor
Chromatic #	$n^{1-\varepsilon}$	No $n^{1-\varepsilon}$ approx

Unique Games Conjecture (Khot 2002): A widely-believed conjecture that (if true) would tighten many inapproximability results: e.g., showing Vertex Cover cannot be approximated below 2.

General TSP is Inapproximable

Theorem

For general (non-metric) TSP, no polynomial-time α -approximation exists for any constant α , unless $P = NP$.

Proof. Reduction from Hamiltonian Cycle.

Given graph $G = (V, E)$, build TSP instance:

$$d_{uv} = \begin{cases} 1 & (u, v) \in E \\ \alpha n + 1 & (u, v) \notin E \end{cases}$$

- If G has a Ham. cycle: OPT tour has cost n .
- If G has no Ham. cycle: every tour uses ≥ 1 non-edge, costing $\geq n - 1 + (\alpha n + 1) > \alpha n$.

A α -approximation algorithm would distinguish cost n from cost $> \alpha n$: solving Ham. Cycle. ✓

Consequence: The triangle inequality is *necessary* for any meaningful TSP approximation.